IA Companion Project

Technical Documentation Report

Raspberry Pi AI Companion with Voice and Sensor Integration

Prepared for Engineering and Integration Review

Date: April 16, 2026

Abstract

This report presents the full technical documentation of the IA Companion system, a Raspberry Pi based intelligent assistant that combines local speech processing, local large language model inference, hardware sensor integration, and orientation-based educational interaction modes. The implemented pipeline performs end-to-end voice interaction through Voice Activity Detection (VAD), speech-to-text (STT), prompt-based language generation (LLM), and text-to-speech (TTS) synthesis with remote audio playback. The architecture supports real-time gyroscope-driven face logic, where orientation triggers spoken level announcements or rules while the assistant remains in a stable general STEM question-answering mode. The system includes GPIO button and light sensor integration and is designed for educational game interoperability with Unreal Engine based mini-games. This document describes hardware and software architecture, processing flow, synchronization and concurrency safeguards, configuration strategy, limitations, and next-step improvements.

Contents

1 Introduction **3**
1.1 Project Motivation . . . 3
1.2 Goal of Building an AI Companion Device . . . 3

2 System Overview **3**
2.1 Hardware Overview . . . 3
2.2 Software Overview . . . 3

3 Hardware Architecture **3**
3.1 Raspberry Pi as Edge Control Node . . . 3
3.2 GPIO Sensors . . . 4
3.3 Light Sensors . . . 4
3.4 Buttons . . . 4
3.5 Gyroscope . . . 4
3.6 Speaker Path . . . 4

4 Software Architecture **4**
4.1 Pipeline Description . . . 4
4.2 Component Breakdown . . . 4
4.3 File Responsibilities . . . 5

5 Voice Processing Pipeline **5**
5.1 VAD Stage . . . 5
5.2 STT Stage . . . 5
5.3 LLM Stage . . . 5
5.4 TTS Stage . . . 5
5.5 Pipeline Diagram . . . 6

6 Sensor Integration **6**
6.1 GPIO Architecture 6
6.2 Light Sensor Usage 6
6.3 Button Input 6
6.4 Gyroscope Orientation Detection 6

7 Orientation-Based Mode System **6**
7.1 Mode Mapping 6
7.2 Runtime Switching Logic 7

8 Game Integration **7**

9 Concurrency and System Reliability **7**
9.1 Synchronization Improvements 7
9.2 Reliability Characteristics 7

10 Environment Configuration **8**

11 Current Limitations **8**

12 Future Improvements **8**

13 Conclusion **9**

1 Introduction

1.1 Project Motivation

The IA Companion project was created to provide an embedded, low-latency, and privacy-preserving intelligent assistant platform that can operate with local inference and local hardware interfaces. By avoiding reliance on cloud-first voice services, the design improves controllability, reduces recurring service dependencies, and supports offline-capable educational deployments.

1.2 Goal of Building an AI Companion Device

The main goal is to build an interactive AI companion that:

- Listens to spoken user input through a microphone.
- Transcribes speech in real time.
- Generates context-aware responses using a local or hosted Ollama model endpoint.
- Speaks responses through a Raspberry Pi speaker path.
- Reacts to physical orientation and sensor states to switch educational behavior modes.

2 System Overview

2.1 Hardware Overview

The hardware platform is centered on a Raspberry Pi integrated with:

- GPIO-connected push buttons.
- GPIO light sensors.
- Gyroscope orientation source exposing front, left, right, and back states.
- Speaker output path (including Bluetooth-capable playback setups).

2.2 Software Overview

The software stack is implemented in Python and organized into modules for audio capture, VAD, STT, LLM communication, TTS generation, and playback transport. The assistant runtime orchestrates these modules and includes a concurrent orientation polling mechanism that plays spoken mode announcements and rules in real time.

3 Hardware Architecture

3.1 Raspberry Pi as Edge Control Node

The Raspberry Pi acts as the sensor and playback endpoint. It hosts interfaces that expose sensor telemetry and receives synthesized audio payloads from the assistant runtime.

3.2 GPIO Sensors

GPIO channels are used for low-latency digital and analog interfacing. The system reads button states and light-related measurements for game and interaction logic.

3.3 Light Sensors

Light sensors provide ambient and directional optical information. This data supports the optics learning mode and can be used for gameplay mechanics requiring light detection and redirection behavior.

3.4 Buttons

Buttons provide direct user input for control actions, interaction confirmation, and possible gameplay events.

3.5 Gyroscope

The gyroscope exposes orientation classes:

- front
- left
- right
- back

These classes are consumed by the assistant to play face-specific level announcements and game rules.

3.6 Speaker Path

Generated audio is transmitted to the Raspberry Pi playback service, with WebSocket preferred for active streaming and HTTP fallback when needed.

4 Software Architecture

4.1 Pipeline Description

The primary software execution path is:

$$\text{Microphone} \rightarrow \text{VAD} \rightarrow \text{STT} \rightarrow \text{LLM} \rightarrow \text{TTS} \rightarrow \text{Speaker}$$

4.2 Component Breakdown

Core modules:

- assistant.py: Main orchestrator and runtime loop.
- audio/recorder.py: Audio capture and utterance segmentation.

- vad/silero_vad.py: Speech probability inference and threshold decision.
- stt/whisper_engine.py: Faster-Whisper based transcription.
- llm/ollama_client.py: Prompt and generation requests to Ollama endpoint.
- tts/piper_engine.py: Piper synthesis plus WebSocket/HTTP/local playback paths.
- utils/audio_utils.py: Audio conversion and playback helpers.
- config.py: Environment loading and typed runtime configuration.
- listen.py: Sensor debugging utility for GPIO endpoint telemetry.

4.3 File Responsibilities

The current implementation separates concerns by domain: acquisition, inference, transport, and configuration. This modular design improves maintainability and allows iterative substitution of models and transports without rewriting the entire runtime.

5 Voice Processing Pipeline

5.1 VAD Stage

Silero VAD estimates speech probability from incoming audio windows and identifies speech boundaries. This limits downstream processing to meaningful utterance segments and reduces unnecessary STT/LLM cost.

5.2 STT Stage

The STT stage uses Faster-Whisper to decode segmented speech into text. Configuration favors low-latency parameters for interactive response.

5.3 LLM Stage

The LLM stage sends user text and a stable general STEM system prompt context to the Ollama generate endpoint. The response is returned as plain text and immediately forwarded to synthesis.

5.4 TTS Stage

Piper produces a WAV response from text. The runtime then attempts WebSocket playback first; if unavailable, it falls back to HTTP POST playback, and if remote playback is not configured, it plays audio locally.

5.5 Pipeline Diagram

Figure 1: End-to-end voice processing and playback pipeline.

6 Sensor Integration

6.1 GPIO Architecture

Sensor data is exposed through a Raspberry Pi endpoint and consumed by polling logic. This architecture decouples hardware sampling from assistant inference while preserving near-real-time control signals.

6.2 Light Sensor Usage

Light sensor values support optical educational interactions and can guide experiment-style responses or game state decisions.

6.3 Button Input

Button states can be used to trigger gameplay interactions, control state transitions, and provide direct tactile feedback in educational scenarios.

6.4 Gyroscope Orientation Detection

The orientation field is monitored continuously. Valid values are normalized and mapped to educational modes. This mapping drives spoken announcements and game-rule guidance.

7 Orientation-Based Mode System

7.1 Mode Mapping

Orientation	**Mode**	**Behavior**
front	Logic Gate Level	Announces activation and then speaks Logic Gates rules.
right	Mirror Level	Announces activation and then speaks Reflection rules.
back	Silent	No TTS output.
left	Silent	No TTS output.

7.2 Runtime Switching Logic

When the detected orientation changes:

1. Poll sensor endpoint and parse gyro orientation.
2. Normalize orientation string.
3. Lookup matching mode profile.
4. Speak level announcement or game rules through TTS.

The LLM prompt remains unchanged during face transitions so the assistant stays in a consistent STEM assistance mode.

8 Game Integration

The system is designed to interoperate with Unreal Engine educational mini-games. Sensor readings from buttons, light sensors, and orientation can be used as gameplay inputs, while the AI assistant provides contextual explanations or hints for each active level. Current mapping already aligns with game-oriented educational domains:

- STEM tutoring on front orientation.
- Logical Systems Gateway challenge on left orientation.
- Optics and light-sensor experimentation on right orientation.
- Future expansion reserved for back orientation.

9 Concurrency and System Reliability

9.1 Synchronization Improvements

Two lock-based safeguards were introduced:

- LLM request lock: protects prompt access and generation in concurrent runtime scenarios.
- TTS lock: prevents overlapping audio playback between face announcements/rules and assistant replies.

9.2 Reliability Characteristics

The gyro poller runs in a daemon thread and continues operation across transient endpoint failures, logging warnings and retrying based on configured polling intervals. This approach improves robustness for real-world sensor intermittency.

10 Environment Configuration

Configuration is loaded from environment variables via .env parsing in config.py. Key runtime parameters include:

- Audio and VAD parameters: sample rate, chunk duration, speech thresholds.
- STT parameters: Whisper model, device, compute type.
- LLM parameters: Ollama URL, model, keep alive, timeout, base system prompt.
- Gyro parameters: URL, polling interval, request timeout.
- TTS and playback parameters: Piper binary/model/config, WebSocket and HTTP playback settings.

Important recently added keys:

- VA_GYRO_URL
- VA_GYRO_POLL_INTERVAL
- VA_GYRO_TIMEOUT

11 Current Limitations

- Level 3 remains a placeholder mode and does not yet implement full educational logic.
- README baseline documentation does not yet include all orientation-mode implementation details.
- Orientation switching currently relies on immediate value changes and may benefit from debounce stability filtering.

12 Future Improvements

- Move per-level prompt text into environment variables for no-code tuning.
- Add orientation debounce and temporal filtering to reduce accidental mode flips.
- Add startup health checks for sensor and playback endpoints.
- Add tests for orientation normalization, mapping, and spoken rules behavior.
- Add optional per-face debounce windows to reduce accidental rapid re-announcements.

13 Conclusion

The IA Companion demonstrates a practical embedded AI assistant architecture that combines local speech intelligence, local language generation, hardware sensor integration, and educational mode adaptation. The implemented orientation-driven announcement and rules system establishes a strong foundation for physically interactive learning experiences while preserving a stable STEM assistant behavior. With incremental reliability enhancements and expanded game content, the platform can evolve into a robust multi-domain educational companion suitable for classrooms, labs, and interactive demonstrations.

Appendix A: Representative Code Snippets

A.1 Face-Driven Announcement Logic

```
LEVEL_BY_ORIENTATION = {
    "front": (
        "Logic gate level activated. "
        "Logic Gates: Press the buttons to send signals through the logic "
        "circuit. Your goal is to activate the final AND gate."
    ),
    "right": (
        "Mirror level activated. "
        "Reflection: Cover the light sensor to send a signal that rotates "
        "the mirrors. Adjust all three mirrors correctly to guide the light "
        "toward the prism and discover what happens when light passes "
        "through it."
    ),
    "back": "",
    "left": "",
}

if orientation and orientation != last_orientation:
    announcement = LEVEL_BY_ORIENTATION[orientation]
    if announcement.strip():
        with speak_lock:
            tts.speak(announcement)
```

A.2 Architecture Overview Diagram

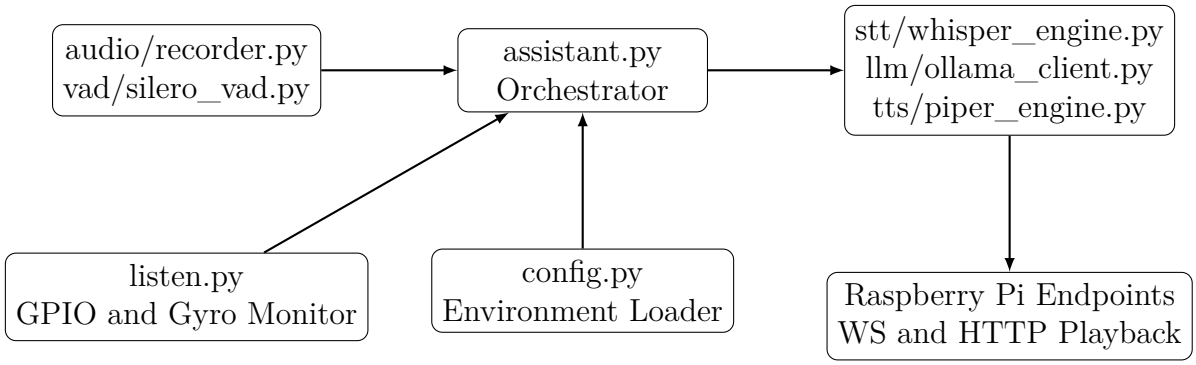

Figure 2: Software architecture relationships across orchestrator, inference modules, configuration, and Raspberry Pi interfaces.